# JSP (Java Server Pages)

**Java Server Pages** (JSP) is a **server**-side programming technology that enables the creation of dynamic, platform-independent method for building Web-based applications. JSP have access to the entire family of **Java** APIs, including the JDBC API to access enterprise databases.

**JSP** technology is used to create web application just like Servlet technology. It can be thought of as an extension to Servlet because it provides more functionality than servlet such as expression language, JSTL, etc.

A JSP page consists of HTML tags and JSP tags. The JSP pages are easier to maintain than Servlet because we can separate designing and development. It provides some additional features such as Expression Language, Custom Tags, etc.

## Advantages of JSP over Servlet

There are many advantages of JSP over the Servlet. They are as follows:

### *1) Extension to Servlet*

JSP technology is the extension to Servlet technology. We can use all the features of the Servlet in JSP. In addition to, we can use implicit objects, predefined tags, expression language and Custom tags in JSP, that makes JSP development easy.

### *2) Easy to maintain*

JSP can be easily managed because we can easily separate our business logic with presentation logic. In Servlet technology, we mix our business logic with the presentation logic.

### *3) Fast Development: No need to recompile and redeploy*

If JSP page is modified, we don't need to recompile and redeploy the project. The Servlet code needs to be updated and recompiled if we have to change the look and feel of the application.

### *4) Less code than Servlet*

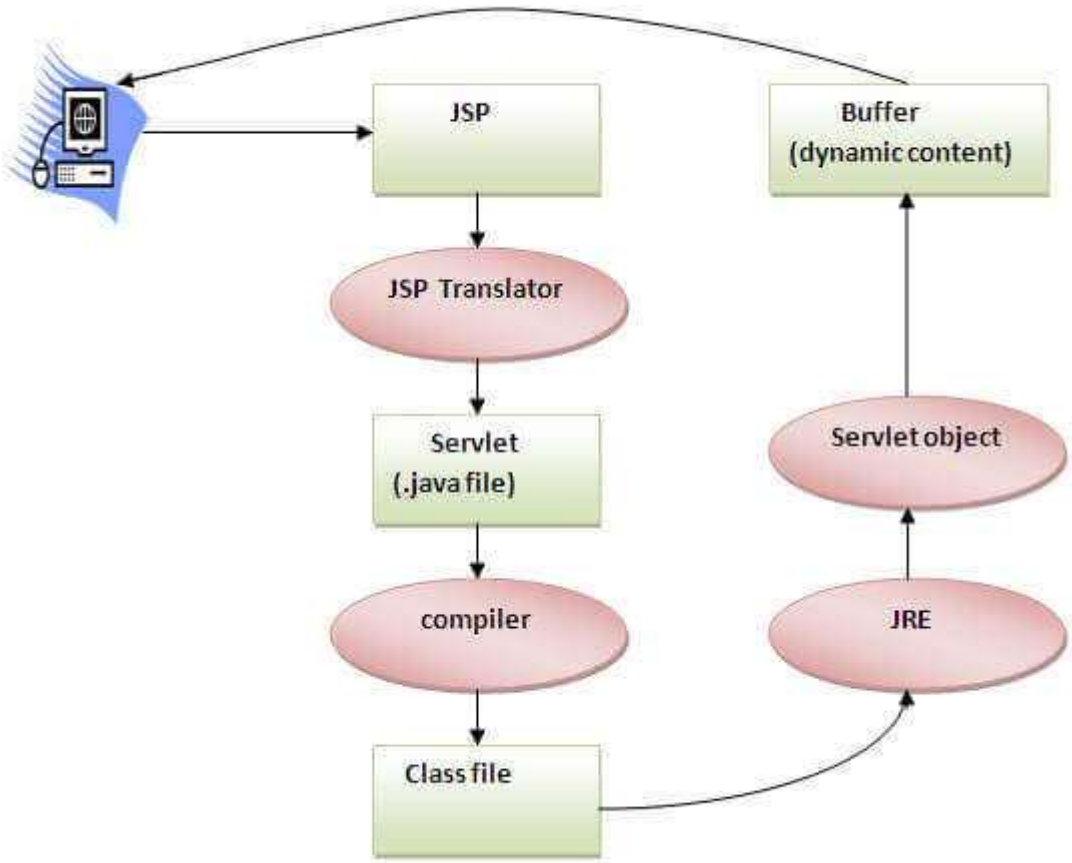
In JSP, we can use many tags such as action tags, JSTL, custom tags, etc. that reduces the code. Moreover, we can use EL, implicit objects, etc.

## The Lifecycle of a JSP Page

The JSP pages follow these phases:

- Translation of JSP Page
- Compilation of JSP Page

- Classloading (the classloader loads class file)
- Instantiation (Object of the Generated Servlet is created).
- Initialization ( the container invokes jspInit() method).
- Request processing ( the container invokes _jspService() method).
- Destroy ( the container invokes jspDestroy() method).




As depicted in the above diagram, JSP page is translated into Servlet by the help of JSP translator. The JSP translator is a part of the web server which is responsible for translating the JSP page into Servlet. After that, Servlet page is compiled by the compiler and gets converted into the class file. Moreover, all the processes that happen in Servlet are performed on JSP later like initialization, committing response to the browser and destroy.

---

## Creating a simple JSP Page

To create the first JSP page, write some HTML code as given below, and save it by .jsp extension. We have saved this file as index.jsp.

**index.jsp**

Let's see the simple example of JSP where we are using the scriptlet tag to put Java code in the JSP page. We will learn scriptlet tag later.

```
<html>
<body>
<% out.print(2*5); %>
</body>
</html>
```

# ➢ JSP Implicit Objects

There are **9 jsp implicit objects**. These objects are *created by the web container* that are available to all the jsp pages.

The available implicit objects are out, request, config, session, application etc.

A list of the 9 implicit objects is given below:

| Object | Type |
|---|---|
| out | JspWriter |
| request | HttpServletRequest |
| response | HttpServletResponse |
| config | ServletConfig |
| application | ServletContext |
| session | HttpSession |
| pageContext | PageContext |
| page | Object |
| exception | Throwable |

## 1) JSP out implicit object

For writing any data to the buffer, JSP provides an implicit object named out. It is the object of JspWriter. In case of servlet you need to write:

1. PrintWriter out=response.getWriter();

   But in JSP, you don't need to write this code.

---

## Example of out implicit object

In this example we are simply displaying date and time.

### index.jsp

```
1. <html>
2. <body>
3. <% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
4. </body>
5. </html>
```

# ➢ JSP directives

The **jsp directives** are messages that tells the web container how to translate a JSP page into the corresponding servlet.

There are three types of directives:

- page directive
- include directive
- taglib directive

### Syntax of JSP Directive

```
1. <%@ directive attribute="value" %>
```

## • JSP page directive

The page directive defines attributes that apply to an entire JSP page.

### Syntax of JSP page directive

```
1. <%@ page attribute="value" %>
```

### Attributes of JSP page directive

- import
- contentType

- extends
- info
- buffer
- language
- isELIgnored
- isThreadSafe
- autoFlush
- session
- pageEncoding
- errorPage
- isErrorPage

## 1)import

The import attribute is used to import class,interface or all the members of a package.It is similar to import keyword in java class or interface.

## Example of import attribute

```
1. <html>
2. <body>
3.
4. <%@ page import="java.util.Date" %>
5. Today is: <%= new Date() %>
6.
7. </body>
8. </html>
```

---

## 2)contentType

The contentType attribute defines the MIME(Multipurpose Internet Mail Extension) type of the HTTP response.The default value is "text/html;charset=ISO-8859-1".

## Example of contentType attribute

```
1. <html>
2. <body>
3.
4. <%@ page contentType=application/msword %>
5. Today is: <%= new java.util.Date() %>
6.
```

```
7. </body>
8. </html>
```

# • Jsp Include Directive

The include directive is used to include the contents of any resource it may be jsp file, html file or text file. The include directive includes the original content of the included resource at page translation time (the jsp page is translated only once so it will be better to include static resource).

## Advantage of Include directive

Code Reusability

### Syntax of include directive

```
1. <%@ include file="resourceName" %>
```

## Example of include directive

In this example, we are including the content of the header.html file. To run this example you must create an header.html file.

```
1. <html>
2. <body>
3.
4. <%@ include file="header.html" %>
5.
6. Today is: <%= java.util.Calendar.getInstance().getTime() %>
7.
8. </body>
9. </html>
```

# • JSP Taglib directive


1. JSP Taglib directive
2. Example of JSP Taglib directive


The JSP taglib directive is used to define a tag library that defines many tags. We use the TLD (Tag Library Descriptor) file to define the tags. In the custom tag section we will use this tag .

### *Syntax JSP Taglib directive*

```
1. <%@ taglib uri="uriofthetaglibrary" prefix="prefixoftaglibrary" %>
```

---

## Example of JSP Taglib directive

In this example, we are using our tag named currentDate. To use this tag we must specify the taglib directive so the container may get information about the tag.

```
1. <html>
2. <body>
3.
4. <%@ taglib uri="http://...../tags" prefix="mytag" %>
5.
6. <mytag:currentDate/>
7.
8. </body>
9. </html>
```

# ➢ JSP Scriptlet tag (Scripting elements)

In JSP, java code can be written inside the jsp page using the scriptlet tag. Let's see what are the scripting elements first.

## JSP Scripting elements

The scripting elements provides the ability to insert java code inside the jsp. There are three types of scripting elements:

- scriptlet tag
- expression tag
- declaration tag

---

### 1. JSP scriptlet tag

A scriptlet tag is used to execute java source code in JSP. Syntax is as follows:

```
1. <%  java source code %>
```

## Example of JSP scriptlet tag

In this example, we are displaying a welcome message.

```
1. <html>
2. <body>
3. <% out.print("welcome to jsp"); %>
4. </body>
5. </html>
```

---

## Example of JSP scriptlet tag that prints the user name

In this example, we have created two files index.html and welcome.jsp. The index.html file gets the username from the user and the welcome.jsp file prints the username with the welcome message.

*File: index.html*

```
1. <html>
2. <body>
3. <form action="welcome.jsp">
4. <input type="text" name="uname">
5. <input type="submit" value="go"><br/>
6. </form>
7. </body>
8. </html>
```

*File: welcome.jsp*

```
1. <html>
2. <body>
3. <%
4. String name=request.getParameter("uname");
5. out.print("welcome "+name);
6. %>
7. </form>
8. </body>
9. </html>
```

# 2. JSP expression tag

The code placed within **JSP expression tag** is *written to the output stream of the response*. So you need not write out.print() to write data. It is mainly used to print the values of variable or method.

## Syntax of JSP expression tag

```
1. <%=  statement %>
```

## Example of JSP expression tag

In this example of jsp expression tag, we are simply displaying a welcome message.

```
1. <html>
2. <body>
3. <%= "welcome to jsp" %>
4. </body>
5. </html>
```

# 3. JSP Declaration Tag

The **JSP declaration tag** is used *to declare fields and methods*.

The code written inside the jsp declaration tag is placed outside the service() method of auto generated servlet.

So it doesn't get memory at each request.

### *Syntax of JSP declaration tag*

The syntax of the declaration tag is as follows:

```
1. <%!  field or method declaration %>
```

## Difference between JSP Scriptlet tag and Declaration tag

| Jsp Scriptlet Tag | Jsp Declaration Tag |
|---|---|
| The jsp scriptlet tag can only declare variables not methods. | The jsp declaration tag can declare variables as well as methods. |
| The declaration of scriptlet tag is placed inside the _jspService() method. | The declaration of jsp declaration tag is placed outside the _jspService() method. |

# ➢ JSP Action Tags

There are many JSP action tags or elements. Each JSP action tag is used to perform some specific tasks.

The action tags are used to control the flow between pages and to use Java Bean. The Jsp action tags are given below.

| JSP Action Tags | Description |
|---|---|
| jsp:forward | forwards the request and response to another resource. |
| jsp:include | includes another resource. |
| jsp:useBean | creates or locates bean object. |
| jsp:setProperty | sets the value of property in bean object. |
| jsp:getProperty | prints the value of property of the bean. |
| jsp:plugin | embeds another components such as applet. |
| jsp:param | sets the parameter value. It is used in forward and include mostly. |
| jsp:fallback | can be used to print the message if plugin is working. It is used in jsp:plu |

The jsp:useBean, jsp:setProperty and jsp:getProperty tags are used for bean development. So we will see these tags in bean developement.

## • jsp:forward action tag

The jsp:forward action tag is used to forward the request to another resource it may be jsp, html or another resource.

### Syntax of jsp:forward action tag without parameter

```
1. <jsp:forward page="relativeURL | <%= expression %>" />
```

### Syntax of jsp:forward action tag with parameter

```
1. <jsp:forward page="relativeURL | <%= expression %>">
2. <jsp:param name="parametername" value="parametervalue | <%=expression%>" />
3. </jsp:forward>
```

---

## Example of jsp:forward action tag without parameter

In this example, we are simply forwarding the request to the printdate.jsp file.

### index.jsp

```
1. <html>
2. <body>
3. <h2>this is index page</h2>
4.
5. <jsp:forward page="printdate.jsp" />
6. </body>
7. </html>
```

### printdate.jsp

```
1. <html>
2. <body>
3. <% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
4. </body>
5. </html>
```

# • jsp:include action tag

The **jsp:include action tag** is used to include the content of another resource it may be jsp, html or servlet.

The jsp include action tag includes the resource at request time so it is **better for dynamic pages** because there might be changes in future.

The jsp:include tag can be used to include static as well as dynamic pages.

---

## Advantage of jsp: include action tag

Code reusability **: We can use a page many times such as including header and footer pages in all pages. So it saves a lot of time.**

## Difference between jsp include directive and include action

| JSP include directive | JSP include action |
|---|---|
| includes resource at translation time. | includes resource at request time. |
| better for static pages. | better for dynamic pages. |
| includes the original content in the generated servlet. | calls the include method. |

### Syntax of jsp:include action tag without parameter

```
1. <jsp:include page="relativeURL | <%= expression %>" />
```

### Syntax of jsp:include action tag with parameter

```
1. <jsp:include page="relativeURL | <%= expression %>">
2. <jsp:param name="parametername" value="parametervalue | <%=expression%>" />
3. </jsp:include>
```

---

## Example of jsp:include action tag without parameter

In this example, index.jsp file includes the content of the printdate.jsp file.

*File: index.jsp*

```
<h2>this is index page</h2>
<jsp:include page="printdate.jsp" />
<h2>end section of index page</h2>
```

*File: printdate.jsp*

```
1. <% out.print("Today is:"+java.util.Calendar.getInstance().getTime()); %>
```

# Custom Tags in JSP

**Custom tags** are user-defined tags. They eliminates the possibility of scriptlet tag and separates the business logic from the JSP page.

The same business logic can be used many times by the use of custom tag.

## Advantages of Custom Tags

The key advantages of Custom tags are as follows:

1. **Eliminates the need of scriptlet tag** The custom tags eliminates the need of scriptlet tag which is considered bad programming approach in JSP.
2. **Separation of business logic from JSP** The custom tags separate the the business logic from the JSP page so that it may be easy to maintain.
3. **Re-usability** The custom tags makes the possibility to reuse the same business logic again and again.

## Syntax to use custom tag

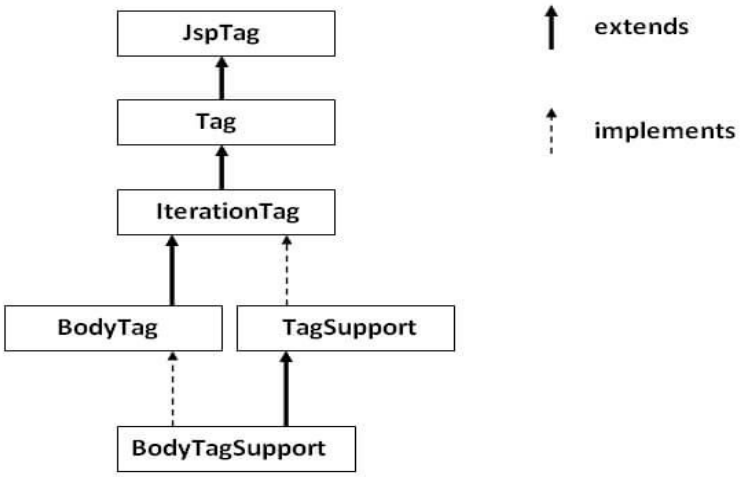
There are two ways to use the custom tag. They are given below:

1. **<prefix:tagname** attr1=value1....attrn=valuen **/>**
1. **<prefix:tagname** attr1=value1....attrn=valuen **>**
2. body code
3. **</prefix:tagname>**

---

# JSP Custom Tag API

The javax.servlet.jsp.tagext package contains classes and interfaces for JSP custom tag API. The JspTag is the root interface in the Custom Tag hierarchy.




---

## • JspTag interface

The JspTag is the root interface for all the interfaces and classes used in custom tag. It is a marker interface.

---

## • Tag interface

The Tag interface is the sub interface of JspTag interface. It provides methods to perform action at the start and end of the tag.

### Fields of Tag interface

There are four fields defined in the Tag interface. They are:

| Field Name | Description |
|---|---|
| **public static int EVAL_BODY_INCLUDE** | it evaluates the body content. |
| **public static int EVAL_PAGE** | it evaluates the JSP page content after the |
| **public static int SKIP_BODY** | it skips the body content of the tag. |

| | |
|---|---|
| **public static int SKIP_PAGE** | it skips the JSP page content after the custo |

### Methods of Tag interface

The methods of the Tag interface are as follows:

| Method Name | Description |
|---|---|
| **public void setPageContext(PageContext pc)** | it sets the given PageContext object. |
| **public void setParent(Tag t)** | it sets the parent of the tag handler. |
| **public Tag getParent()** | it returns the parent tag handler object. |
| **public int doStartTag()throws JspException** | it is invoked by the JSP page implementation object. Th should override this method and define the business lo at the start of the tag. |
| **public int doEndTag()throws JspException** | it is invoked by the JSP page implementation object. Th should override this method and define the business lo at the end of the tag. |
| **public void release()** | it is invoked by the JSP page implementation object to |

---

## • IterationTag interface

The IterationTag interface is the sub interface of the Tag interface. It provides an additional method to reevaluate the body.

### Field of IterationTag interface

There is only one field defined in the IterationTag interface.

- **public static int EVAL_BODY_AGAIN** it reevaluates the body content.

### Method of Tag interface

There is only one method defined in the IterationTag interface.

- **public int doAfterBody()throws JspException** it is invoked by the JSP page implementation object after the evaluation of the body. If this method returns EVAL_BODY_INCLUDE, body content will be reevaluated, if it returns SKIP_BODY, no more body cotent will be evaluated.

---

## • TagSupport class

The TagSupport class implements the IterationTag interface. It acts as the base class for new Tag Handlers. It provides some additional methods also.

---